Objektumok kapcsolatai

Gregorics Tibor

gt@inf.elte.hu

http://people.inf.elte.hu/gt/oep

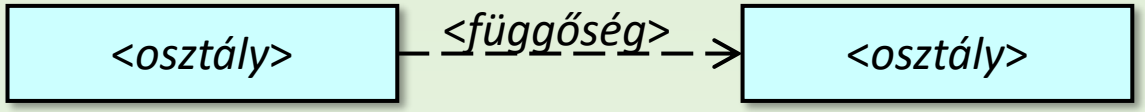
Objektum-kapcsolatok fajtái

- Amikor objektumok egymással kommunikálnak (szinkron vagy aszinkron módon egymás metódusait hívják, egyik a másiknak szignált küld, esetleg közvetlenül a másik adattagjain végeznek műveletet), akkor kapcsolat alakul ki közöttük.
- A kapcsolat fajtája lehet:
  - Függőség (*dependency*)
  - Asszociáció (*association*) vagy társítás
  - Aggregáció (*shared aggregation*) vagy tartalmazás
  - Kompozíció (*composite aggregation*) vagy szigorú tartalmazás
  - Származtatás vagy öröklődés (*inheritence*)
- Az objektumok közötti kapcsolatokat az osztályaik szintjén ábrázolhatjuk (az osztálydiagram az objektumdiagram absztrakciója).

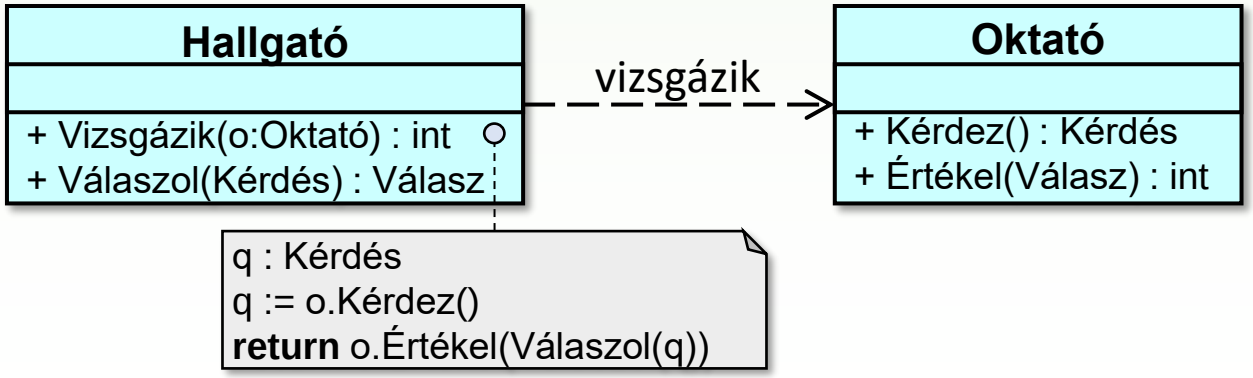
4

Az objektum-orientált nyelvek ismérve a fogalmi szintű absztrakció. (A kapcsolatok leírására azonban csak néhányuknak van nyelvi eleme.)

Függőség

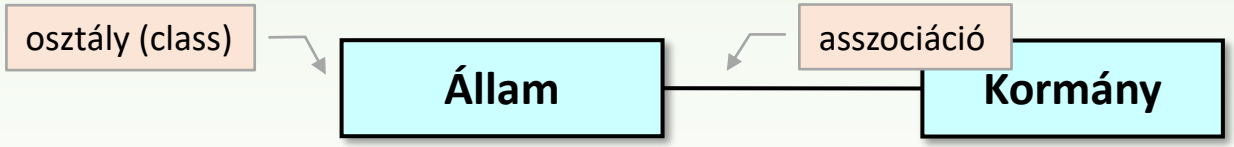


- Egy objektum epizódszerűen (rövid időre) kerül kapcsolatba egy másik objektummal (vagy úgy, hogy a másik objektumot paraméterként kapja meg, vagy maga példányosítja azt), és ennek a másik objektumnak
  - egyik metódusát hívja egy saját metódusából,
  - egy szignált (üzenetet) küld,
  - a hivatkozását adja tovább (pl. kivételkezelésnél),
  - egy lehetséges állapotát (értékét) használja (pl. felsorolt típus értékét.
- Nemcsak objektumok, hanem osztályok között is lehet függőség, amikor egy metódus (ez lehet osztályszintű is) egy másik osztály osztályszintű metódusát hívja.

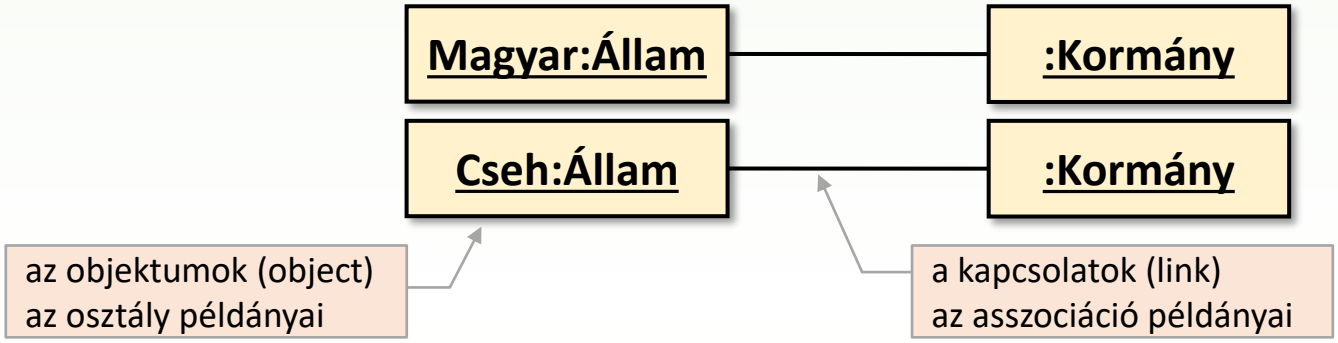


Asszociáció

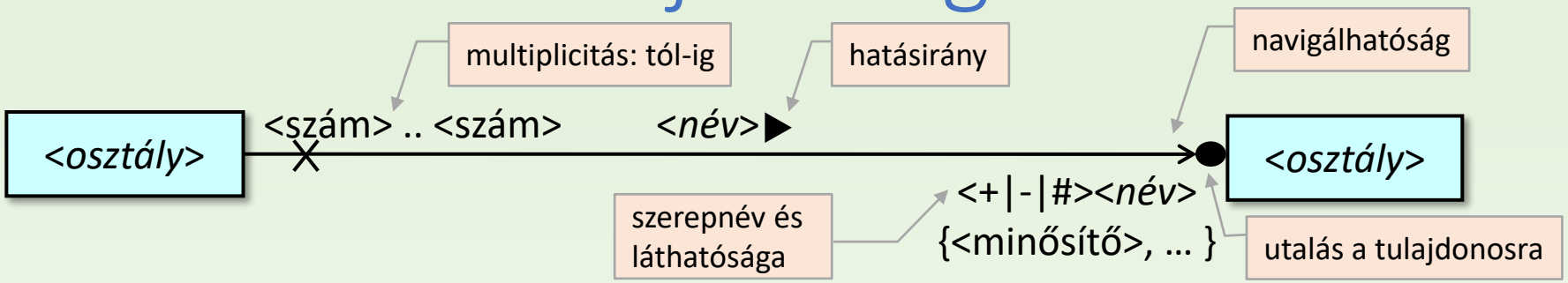
- Az asszociáció az objektumok között hosszabb időszakon keresztül fennálló kapcsolat, amely mentén az objektumok sorozatosan küldenek egymásnak üzeneteket, és ezért a kapcsolatban álló objektumok hivatkozásait a gyors elérés érdekében érdemes külön is eltárolni.
- Matematikai értelemben az asszociáció egy reláció az adott osztályok példányait (objektumait) tartalmazó halmazok direkt szorzatán:
  - egy asszociáció több objektum-kapcsolatot ír le, így ugyanaz az objektum egy asszociáció több kapcsolatában is szerepelhet, de megjelenhet különböző asszociációk kapcsolataiban is.



Az osztálydiagram egy lehetséges példányosítása (felpopulálása):



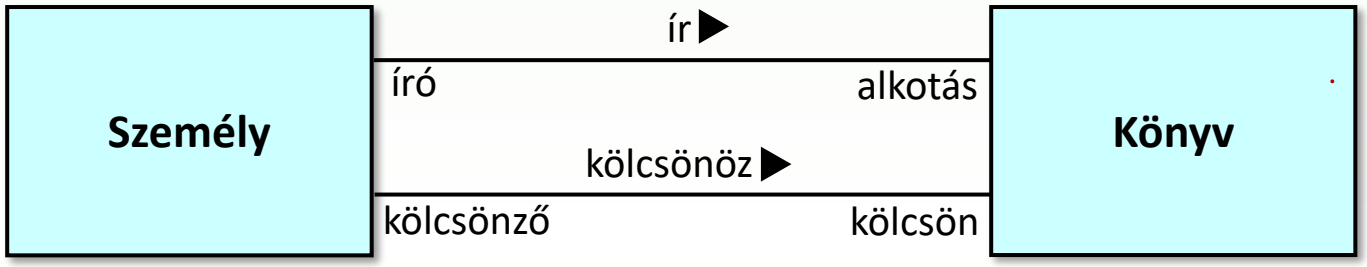
Asszociáció tulajdonságai



- Az asszociáció tulajdonságai az általa leírt kapcsolatokat jellemzik:
  - név: a kapcsolatok közös megnevezése
  - hatásirány: kapcsolódó objektumok közti fogalmi viszony
  - multiplicitás: egy objektumhoz kapcsolható objektumok száma
  - aritás: az egyes kapcsolatokban résztvevő objektumok száma
  - navigálhatóság: a kapcsolatban melyik objektumot kell gyorsan elérni
  - asszociációvég nevek: a kapcsolatban álló objektumok hivatkozási nevei
  - asszociációvégek neveinek láthatósága
  - asszociációvégek neveinek tulajdonosa
  - egy objektumhoz kapcsolódó több objektum által alkotott gyűjtemény minősítése

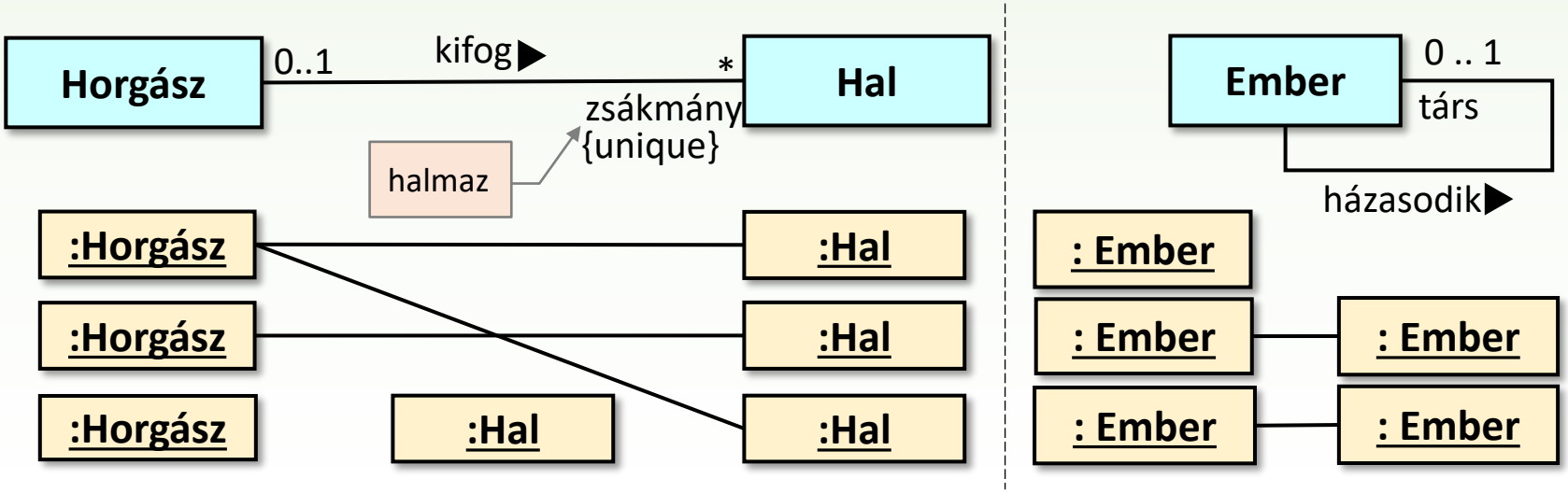
Nevek és a hatásirány

- Egy objektumelvű modell asszociációi és a megoldandó feladat szöveges leírásának mondatai között szoros kapcsolat áll fenn. A modell megértését segíti, ha az asszociációkat, és annak kapcsolataiban megjelenő objektumokat a feladatból származó nevekkel látjuk el:
  - az asszociáció neve egy mondat állítmánya (néha tárgya), a mondat többi (nem állítmány, nem jelző) eleme pedig az asszociációvégeknek adhat nevet (szerepnév).
- A bináris (két objektum kapcsolatát leíró) asszociációk neve mellé rajzolt fekete háromszög hegye az asszociáció hatásiránya
  - ez mindig az asszociációt jellemző mondat alanyát adó objektum felől mutat a másik (sokszor ez a mondat tárgya) irányába. Az alany osztályában többnyire megjelenik az asszociáció nevével megegyező nevű metódus is.



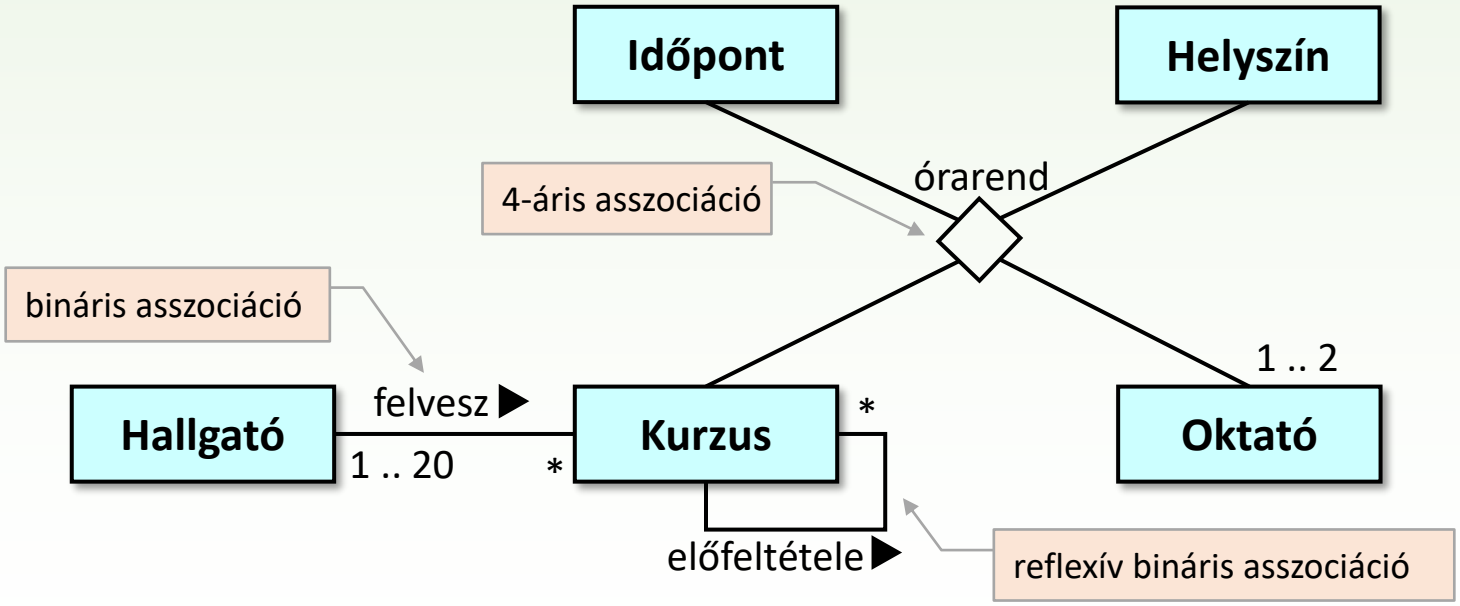
Multiplicitás

- Az asszociáció multiplicitása azt mutatja, hogy az asszociáció alapján egy objektum hány másik objektummal létesíthet egyszerre kapcsolatot. Ez lehet egy természetes szám (alapértelmezés szerint 1), vagy természetes számok min..max intervalluma. (Ha a szám, illetve a max helyén * áll, akkor az az adott érték tetszőleges voltára utal.)
- Előírhatjuk hogy egy objektumhoz kapcsolódó objektumok
  - mind különbözzenek egymástól {unique}
  - megadott sorrendben legyenek felsorolhatók {ordered}



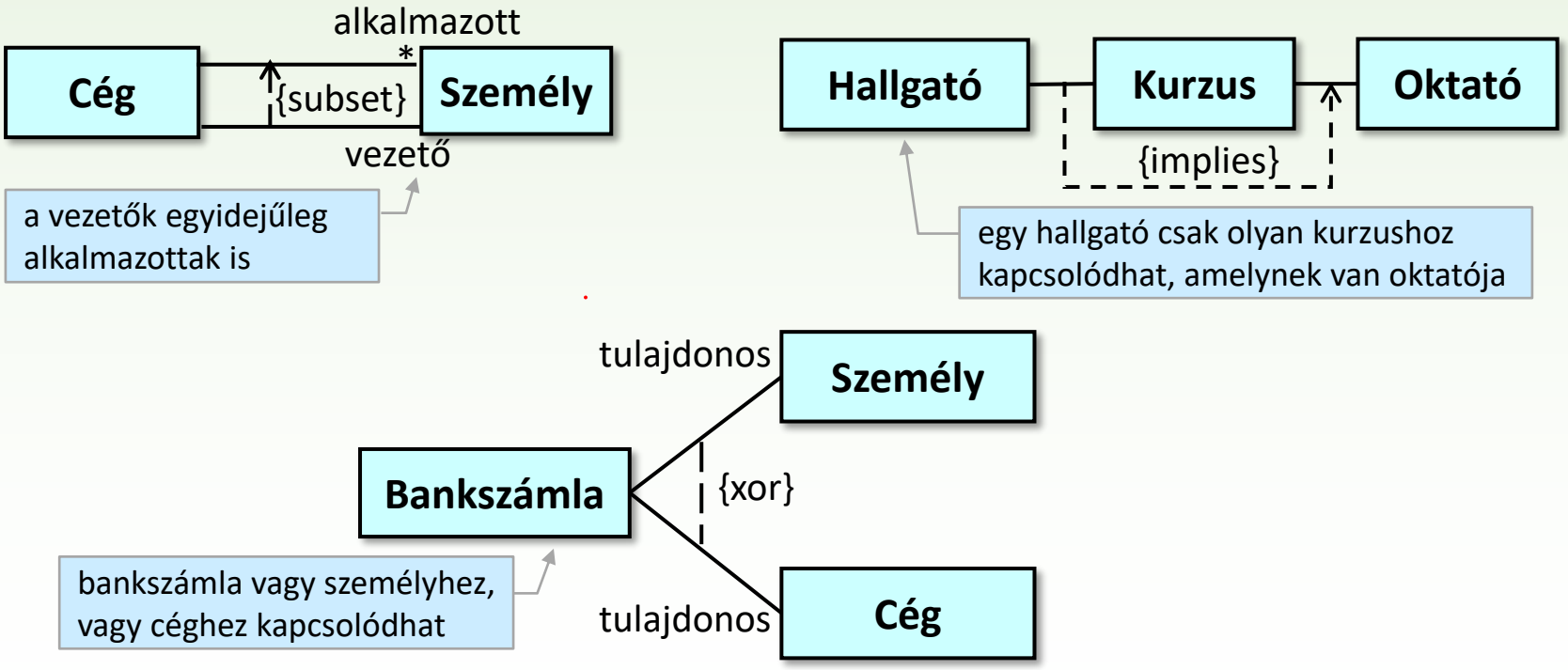
Aritás

- Az asszociáció aritása arra utal, hogy az asszociáció egyetlen kapcsolata hány objektumot köthet össze. (Eddig csak bináris asszociációkra láttunk példákat, ahol egy kapcsolat mindig két objektum között jött létre.)
- Ne keverjük össze az aritás és a multiplicitás fogalmait!



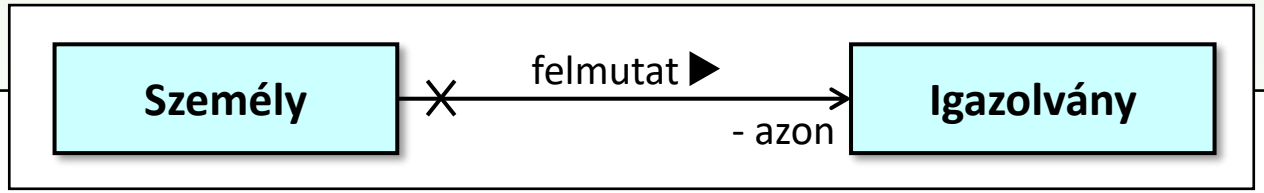
Asszociációk közötti feltételek

- Megadhatunk az asszociációk között logikai feltételeket (subset, and, or, xor, implies, …), amelyek különböző asszociációk kapcsolatai közötti korlátozásokat fogalmazzák meg.



Navigálhatóság

- A navigálhatóság azt jelzi, hogy az asszociáció kapcsolataiban melyik objektumot kell a kapcsolat másik (többi) objektumának hatékonyan elérni. Az asszociáció végén álló jel lehet
  - nyíl: a hatékonyan elérendő objektumok osztályára mutat.
  - „×”: a jelzett osztály objektumait nem kell hatékonyan elérni.
  - jelöletlen: nincs eldöntve vagy lényegtelen a jelzett osztály objektumainak elérhetősége.
- A hatásirány és a navigálhatóság iránya különböző fogalmak: az előbbi a fogalmak megértését segíti, az utóbbi az implementációra ad előírást.



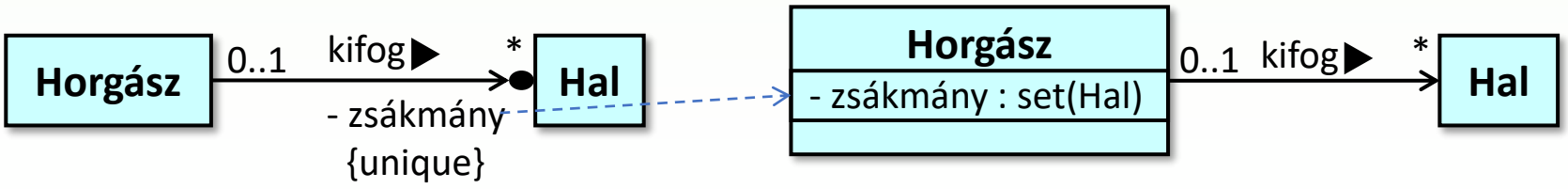

```
class Person {
private:
      IdentityCard *_id;
public:
      IdentityCard* showIdentityCard() const {
            if (nullptr!=_id) return _id
            else throw ESCAPE;
      }
};
```

valamikor értéket kap

megadja a kapcsolódó igazolvány-objektum hivatkozását

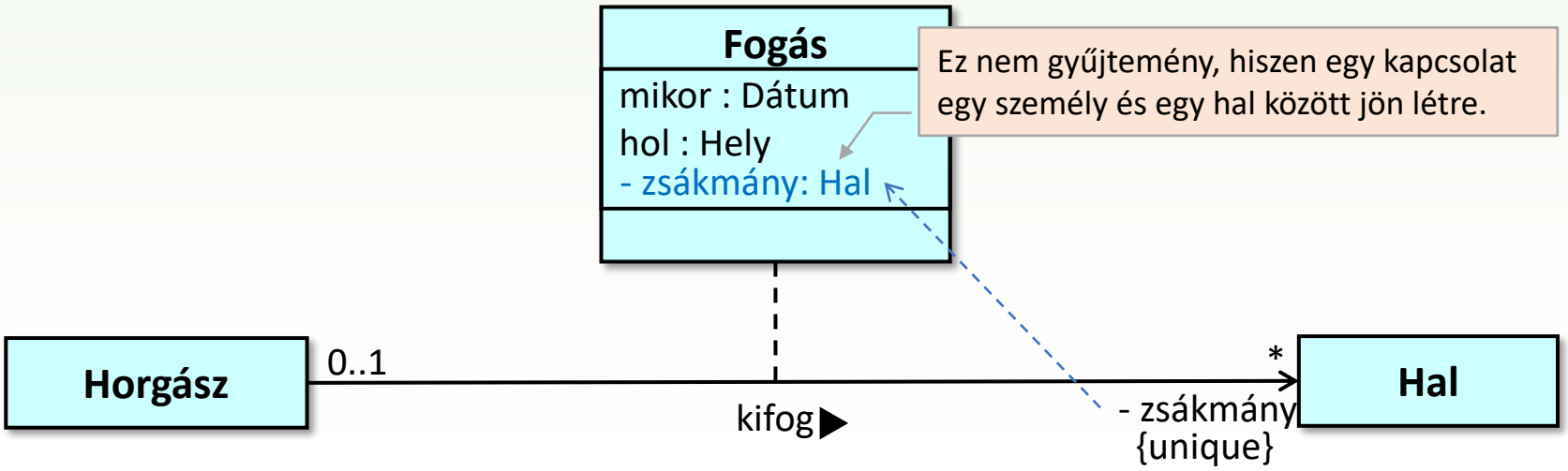
Szerepnév tulajdonosa

- Amikor egy kapcsolatban egy objektumnak el kell érni a másikat, akkor a másik objektum hivatkozását (ennek neve lesz a szerepnév) tárolni kell.
- El kell dönteni, ki legyen a szerepnév tulajdonosa, azaz hol tároljuk a kapcsolat másik objektumának hivatkozását?
  - A kapcsolat másik (többi) objektumában, amelynek a szerepnév egy adattagja lesz. A gyakorlatban ez az elterjedtebb (az UML ezt egy *fekete pöttyel* jelöli az asszociációs vonalnak az adott szerepnév melletti végén, de ezt nem igen használják). Ilyenkor ha a szerepnévhez többszörös multiplicitás tartozik, akkor a szerepnév egy gyűjteményre hivatkozik.
  - Magában a kapcsolatban, amely ezért önálló tárhellyel rendelkezik, amit a kapcsolat objektumai elérnek.
- A szerepnév elérhetőségét a láthatósága befolyásolja.



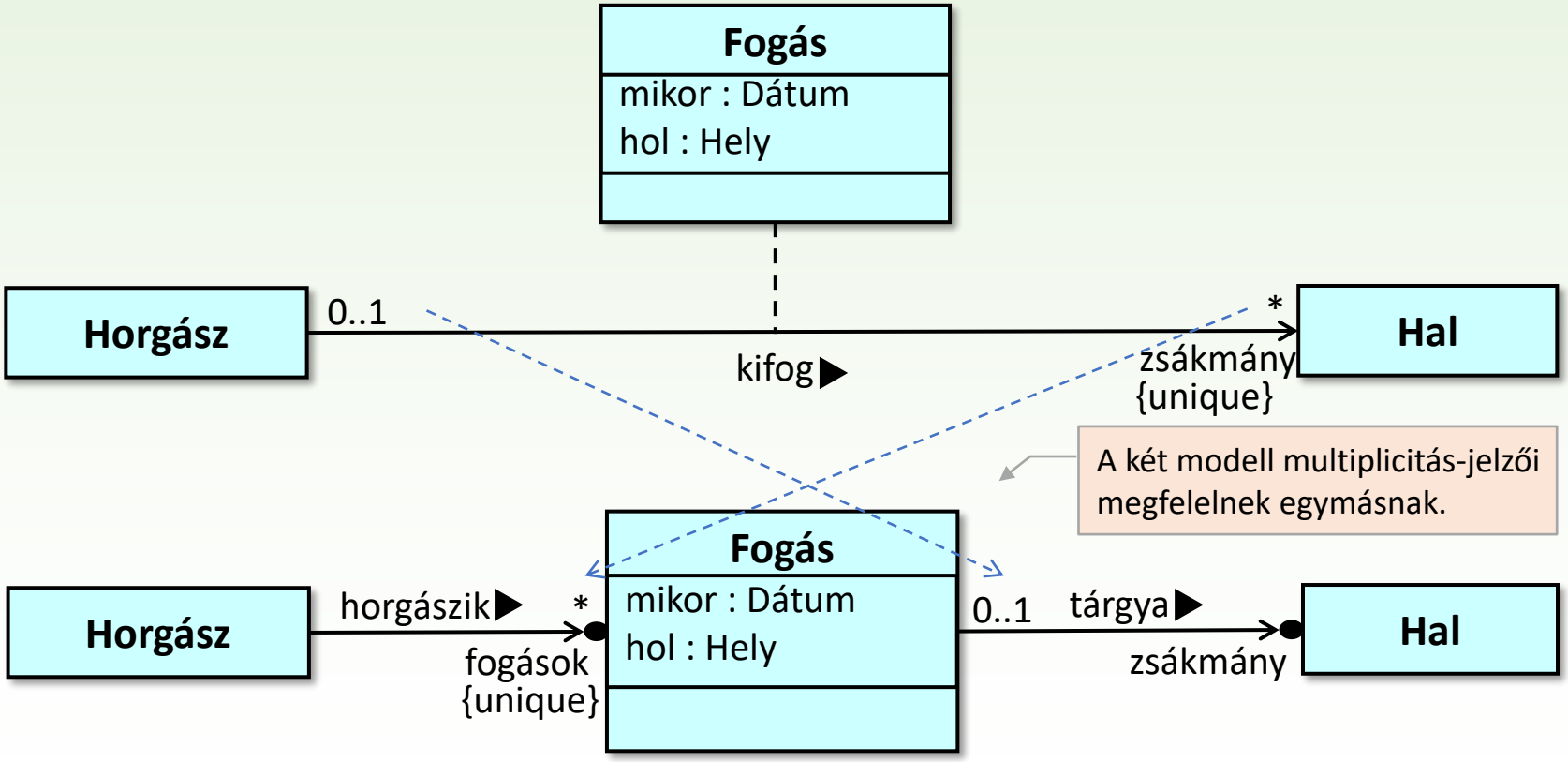
Asszociációs osztály

- Az UML lehetőséget ad arra, hogy az asszociációk kapcsolatainak egyéb tulajdonságait is leírhassuk, és ezeket a kapcsolatok számára létrehozott tárhelyen tároljuk. Ennek leírására az ún. asszociációs osztály szolgál, amelynek a példányai a kapcsolatok, és amelyeket közvetlenül elérnek a kapcsolat által összekötött objektumok.
- Amikor az asszociáció a tulajdonosa az asszociáció egy szerepnevének, akkor ez a szerepnév az asszociációs osztálynak az adattagja, amely az asszociációból példányosított összes kapcsolatnak adattagja.

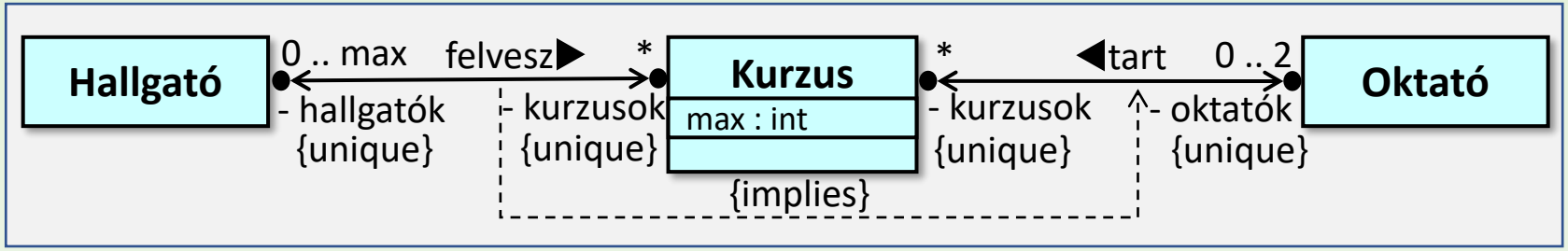


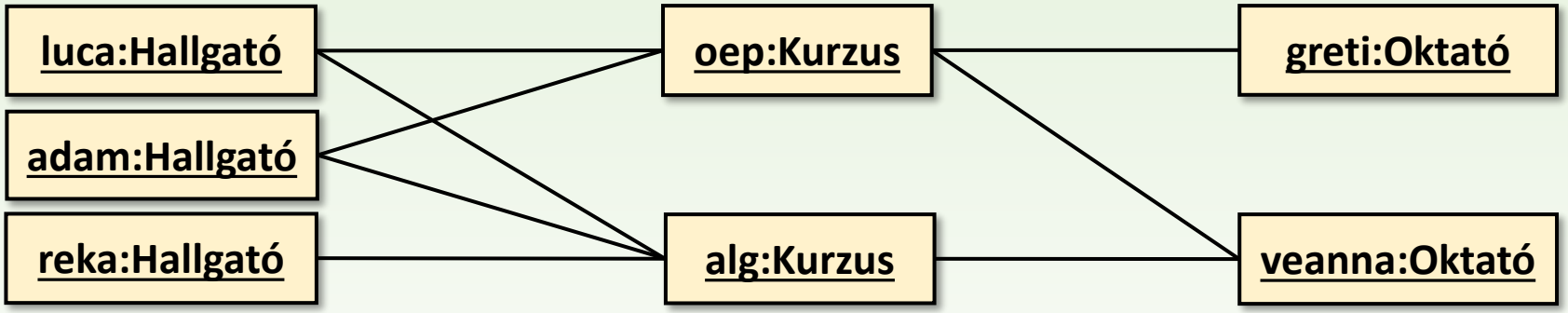
Asszociációs osztály kiküszöbölése

- A programozási nyelvek többsége az asszociációs osztály fogalmát nem támogatja: a kapcsolatokhoz nem rendelnek önálló tárhelyet. Ilyenkor ezt közönséges osztály segítségével helyettesíthetjük.



Példa: Kurzusok






```
Student luca, adam, reka;
Teacher greti, veanna;
Course oep(20), alg(22);

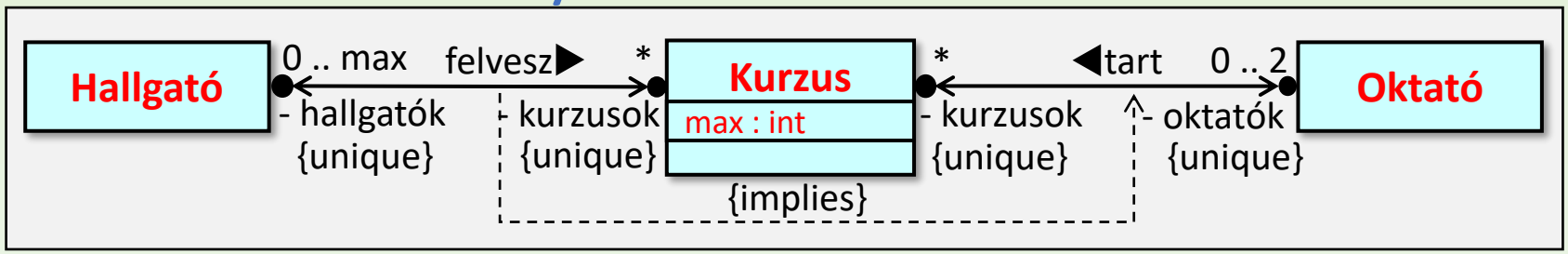
greti.undertakes(&oep);
veanna.undertakes(&oep); veanna.undertakes(&alg);

luca.signs_up(&alg); luca.signs_up(&oep);
adam.signs_up(&alg); adam.signs_up(&oep);
reka.signs_up(&alg);
```

Először az objektumokat példányosítjuk.

A kapcsolatokat az asszociációk nevével azonos nevű metódusok hozzák létre.

Példa osztályai



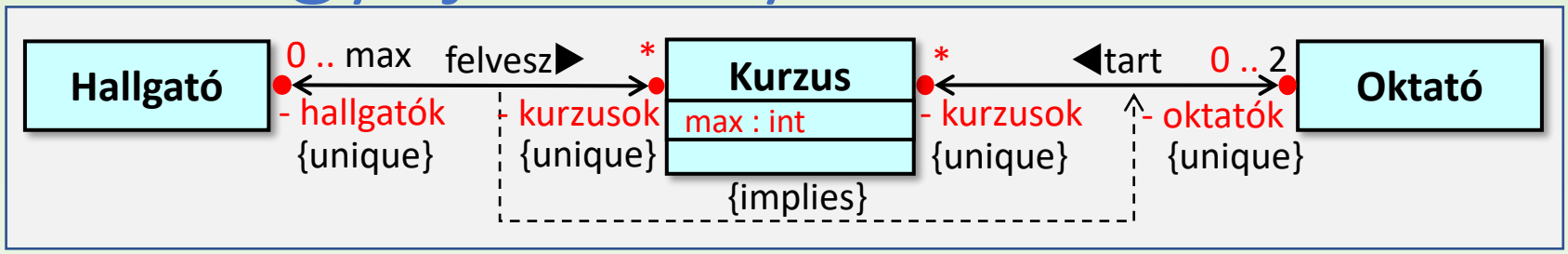

```
class Course {
private:
        int _max;
        …

public:
        Course(int a) : _max (a) { }
};
```

```
class Student {
        …
};
```

```
class Teacher {
        …
};
```

Példa gyűjteményei



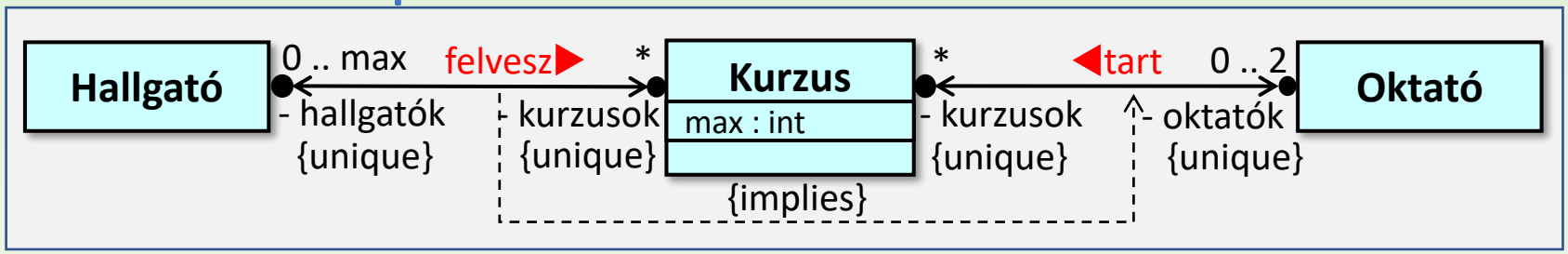
Gyűjtemények tárolják egy objektumhoz kapcsolódó objektumok hivatkozásait. Ez hatékonyan támogatja a navigációt.

```
class Course {
private:
      int _max;
      std::vector<Teacher*> _teachers;
      std::vector<Student*> _students;
public:
      Course(int a) : _max (a) { }
};
```

```
class Student {
private:
      std::vector<Course*> _courses;
public:
      …
};
```

```
class Teacher {
private:
      std::vector<Course*> _courses;
public:
      …
};
```

Példa kapcsolatainak létrehozása



Az asszociáció neve egy ugyanilyen nevű metódust feltételez az asszociációval összekötött osztályokban.

```
class Course {
private:
        int _max;
        std::vector<Teacher*> _teachers;
        std::vector<Student*> _students;
public:
        Course(int a) : _max (a) { }
};
```

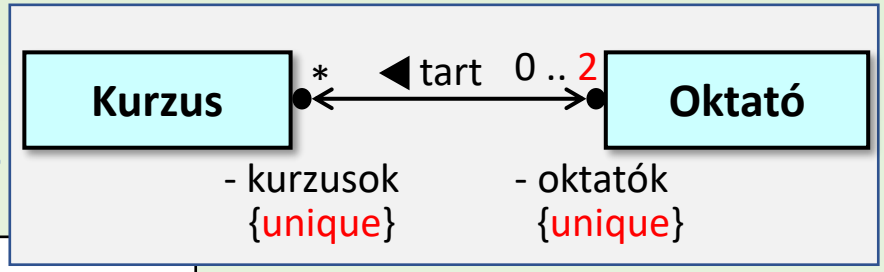
```
class Student {
private:
        std::vector<Course*> _courses;
public:
        void signs_up(Course *pc);
};
```

A felvesz() metódus hoz létre egy újabb hallgató-kurzus kapcsolatot.

```
class Teacher {
private:
        std::vector<Course*> _courses;
public:
        void undertakes(Course *pc);
};
```

A tart() metódus hoz létre egy újabb oktató-kurzus kapcsolatot.

Példa metódusai 1.



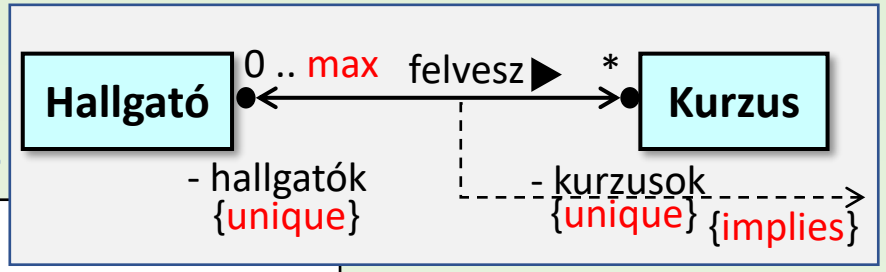

```
class Course {
private:
      int _max;
      std::vector<Teacher*> _teachers; // 0 .. 2
      std::vector<Student*> _students;
public:
      Course(int a) : _max (a) { }
      bool can_lead(Teacher *pt) {
            if (_teachers.size()>=2) return false;
            for (Teacher *p : _teachers) {
                  if ( p==pt ) return false;
            }
            _teachers.push_back(pt);
            return true;
      }
};
```

Itt vizsgáljuk, hogy egy újabb kapcsolattal nem lépjük túl a multiplicitás felső korlátját.

Ez a lineáris keresés vizsgálja meg, hogy az újabb kapcsolattal nem sérülnek-e a unique feltételek.

```
class Teacher {
private:
      std::vector<Course*> _courses;
public:
      void undertakes(Course *pc){
            if ( pc==nullptr ) return;
            if ( pc->can_lead(this) ) {
                  _courses.push_back(pc);
            }
      }
};
```

Példa metódusai 2.




```
class Course {
private:
      int _max;
      std::vector<Teacher*> _teachers; // 0 .. 2
      std::vector<Student*> _students; // 0 .. max
public:
      Course(int a) : _max (a) { }
      bool can_lead(Teacher *pt) { … }
      bool has_teacher() const { return _teachers.size()>0; }
      bool can_sign_up(Student *ps) {
            if ( _students.size()>=unsigned(_max ) ) return false;
            for (Student *p : _students) {
                  if ( p==ps ) return false;
            }
            _students.push_back(ps);
            return true;
      }
};
```

Itt ellenőrizzük a multiplicitás felső korlátját.

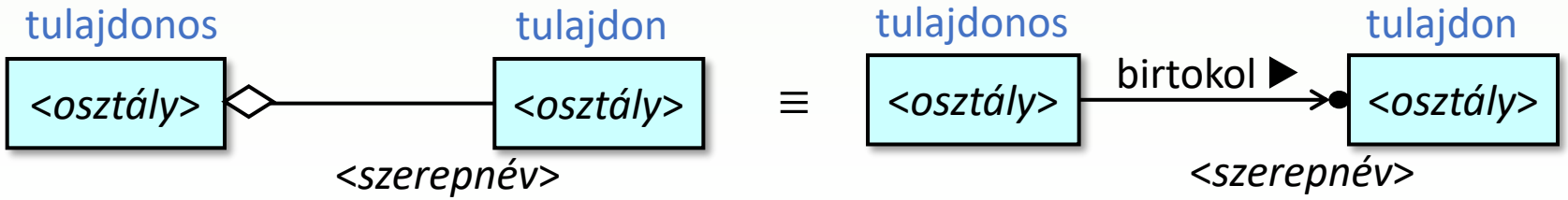
Itt ellenőrizzük az implies feltételt.

Ez a lineáris keresés vizsgálja meg, hogy az újabb kapcsolattal nem sérülnek-e a unique feltételek.

```
class Student {
private:
      std::vector<Course*> _courses;
public:
      void signs_up(Course *pc){
            if ( pc==nullptr  || !pc->has_teacher() )
                  return;
            if ( pc->can_sign_up(this) )
                  _courses.push_back(pc);
      }
};
```

Aggregáció

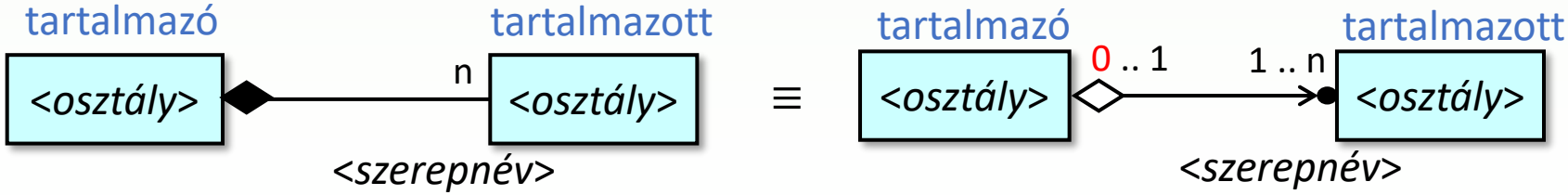
- Birtoklási kapcsolat (tulajdonos-tulajdon) kifejezésére szolgáló bináris asszociáció, ahol a kapcsolatok nem alkothatnak irányított kört (egy objektum még közvetett módon sem lehet önmaga tulajdona).
- Megengedett, hogy
  - a tulajdonos objektumnak ne legyen tulajdon objektuma;
  - egy tulajdon objektum önmagában is létezzen
  - egy tulajdon objektum egyszerre több tulajdonos objektumhoz is tartozzon.
- Megállapodás szerint a tulajdon objektum irányába mindig hatékonyan navigálható úgy, hogy a tulajdonos objektum tárolja el a tulajdonának szerepnevét.



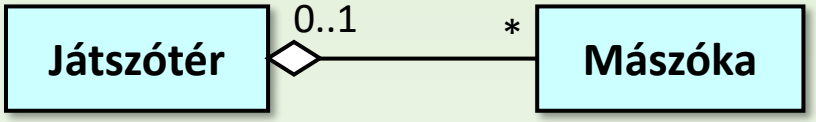
Kompozíció

- Speciális aggregációként az egész-rész (tartalmazó-tartalmazott) kapcsolat kifejezésére szolgáló bináris asszociáció, ahol
  - a tartalmazó objektum nem létezhet tartalmazott objektum nélkül,
  - a tartalmazott objektum egyszerre csak egy objektum része lehet.

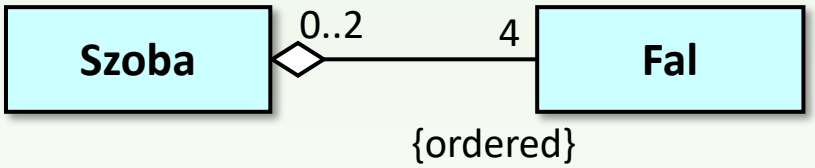
  Implementáláskor a tartalmazott objektumot nem kell fizikailag beágyazni a tartalmazó objektumba : elég csak hivatkozni rá.
- A kompozíció fogalmához sokan az alábbi, a tartalmazott objektumra tett, egyre szigorodó megszorításokat valamelyikét is hozzáértik:
  - van tartalmazója: a tartalmazott önmagában nem létezhet.
  - a tartalmazója nem változik: a tartalmazott objektumot a tartalmazó objektum metódusai példányosítják, és törlik.
  - élettartama azonos a tartalmazójáéval: a tartalmazott objektumot a tartalmazó objektum konstruktora példányosítja, destruktora törli.



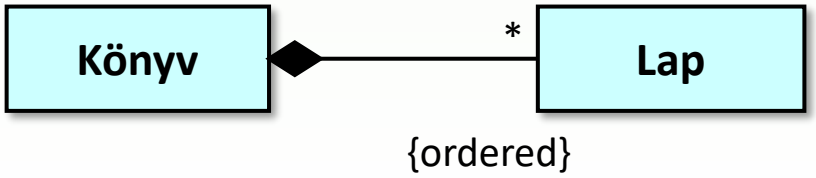
Példák: aggregáció vs. kompozíció



A mászókák a játszótér részei, de egy játszótér mászóka nélkül is játszótér.
(Egy mászóka egyszerre csak egy játszótér tartozéka lehet, de átvihető másikra is, sőt önmagában is használható.)

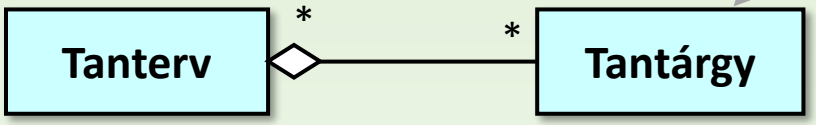


A falak a szobák részei, de ugyanaz a fal egyszerre két szobához is tartozhat.
(Egy szoba kidőlt falakkal talán már nem szoba, viszont egy fal önmagában is fal.)

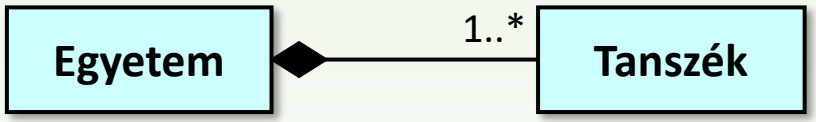


Egy könyv adott számú lapból áll, és a lapjai mindig csak az adott könyvhöz tartozhatnak. (A könyv egy-egy lapja önmagában is használható.)

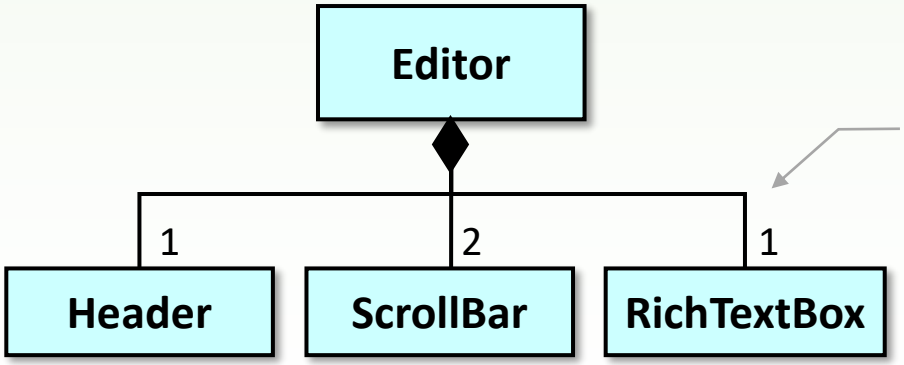
Példák: aggregáció vs. kompozíció



Tanterv nincs tantárgyak nélkül, de egy tantárgy egyszerre több tantervnek is lehet része (sőt tantervhez sem kell tartoznia).



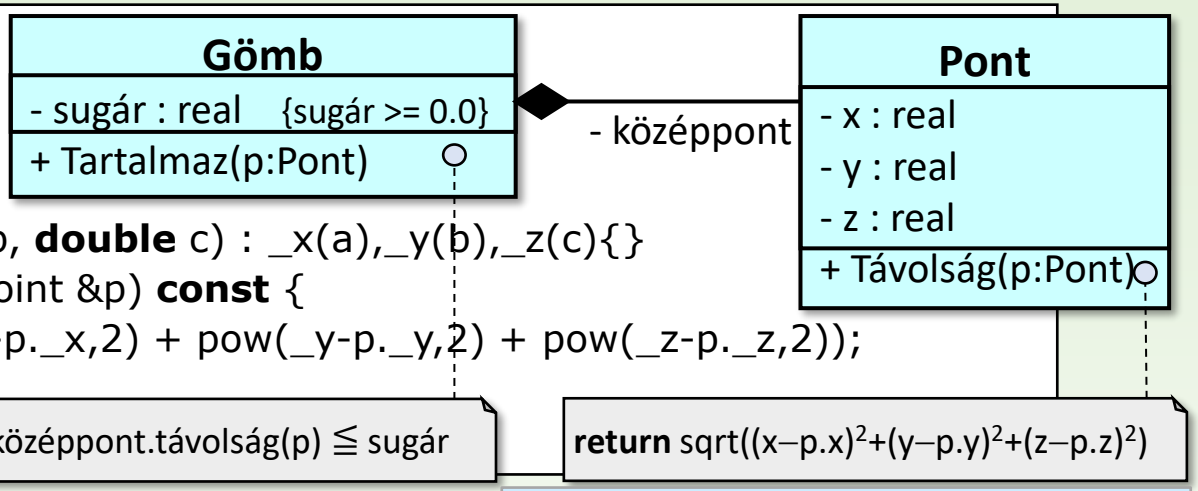
Nincs egyetem tanszékek nélkül (persze tanszék sincs egyetem nélkül). Tanszék nem lehet egyszerre több egyetem tanszéke (sőt nem kerülhet át másik egyetemre, és megszűnik, ha bezárják az egyetemet).



A szerkesztő ablak létrehozásával egyszerre jön létre annak fejléce, gördítősávjai, és szerkesztő felülete is, amelyek majd az ablak megszűnésekor szűnnek meg.

Példa: pont a gömbben (1.)

```
class Point {
private:
    double _x, _y, _z;
public:
    Point(double a, double b, double c) : _x(a),_y(b),_z(c){}
    double distance(const Point &p) const {
        return sqrt(pow(_x-p._x,2) + pow(_y-p._y,2) + pow(_z-p._z,2));
    }
};
```




```
class Sphere{
private:
    Point  _centre;
    double _radius;
public:
    enum Errors{ILLEGAL_RADIUS};
    Sphere(const Point &c, double r): _centre(c), _radius(r) {
        if (_radius<0.0) throw new IllegalRadiusException;
    }
    bool contains(const Point &p) const {
        return _centre.distance(p) <= _radius;
    }
};
```

kompozíció előkészítése lehetne Point *_centre

kompozíció megvalósítása

Számoljuk meg, hogy adott térbeli pontok közül hány található egy adott gömb belsejében:

$$\sum_{pont \in pontok} 1$$

gömb.Tartalmaz(pont)

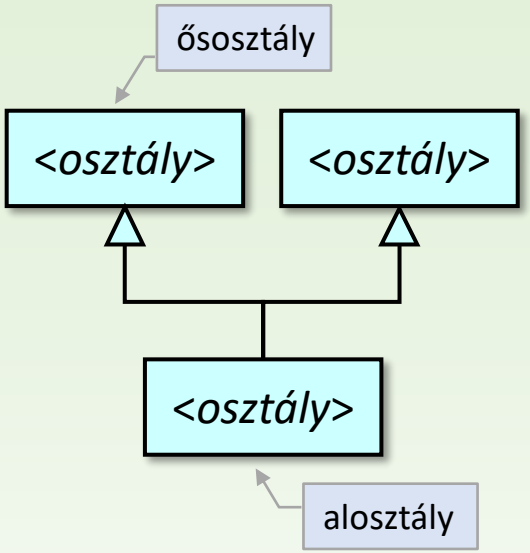
```
Point  p(-12.0, 0.0, 23.0);
Point  c(-12.3, 0.0, 23.4);
Sphere g(c, 1.0);
cout << g.contains(p) << endl;
cout << c.distance(p) << endl;
```

Kapcsolatok létrehozásának helye

- Asszociáció esetén egy kapcsolatot a kapcsolat egyik objektuma hozza létre valamelyik metódusával (ez gyakran az asszociáció nevét viseli, de lehet a konstruktor is). Ez a metódus vagy paraméterként kapja meg a kapcsolat másik (többi) objektumának hivatkozását, vagy maga példányosítja azt (azokat), és gondoskodik a hivatkozásaik eltárolásáról.
- Aggregáció esetén a kapcsolat létrehozása a tulajdonos objektum felelőssége: ennek metódusa helyezi el a tulajdon objektum hivatkozását a tulajdonos erre szolgáló adattagjában.
- Kompozíció esetén többnyire a tartalmazó objektum konstruktora építi ki a kapcsolatot a tartalmazott objektummal.
  - Ha a tartalmazott objektum lecserélhető, akkor erre a tartalmazó objektum egy metódusa szolgál.
  - Szigorúbb értelmezés esetén a tartalmazó objektum konstruktora példányosítja a tartalmazott objektumot, és a destruktora törli.

Származtatás, öröklődés

- Ha egy objektum más objektumokra hasonlít, azaz azokkal megegyező adattagjai és metódusai vannak, más szóval örököli azok tulajdonságait, de ezeket módosíthatja, ki egészítheti, akkor az osztálya a vele hasonló objektumok osztályainak mintájára definiálható, azaz belőlük származtatható.



- A modellezés során kétféle okból használunk származtatást:
  - Általánosítás: már meglévő, egymáshoz hasonló osztályoknak a közös tulajdonságait leíró ősosztályát (szuperosztály) hozzuk létre.
  - Specializálás: egy osztályból származtatással hozunk létre egy alosztályt.

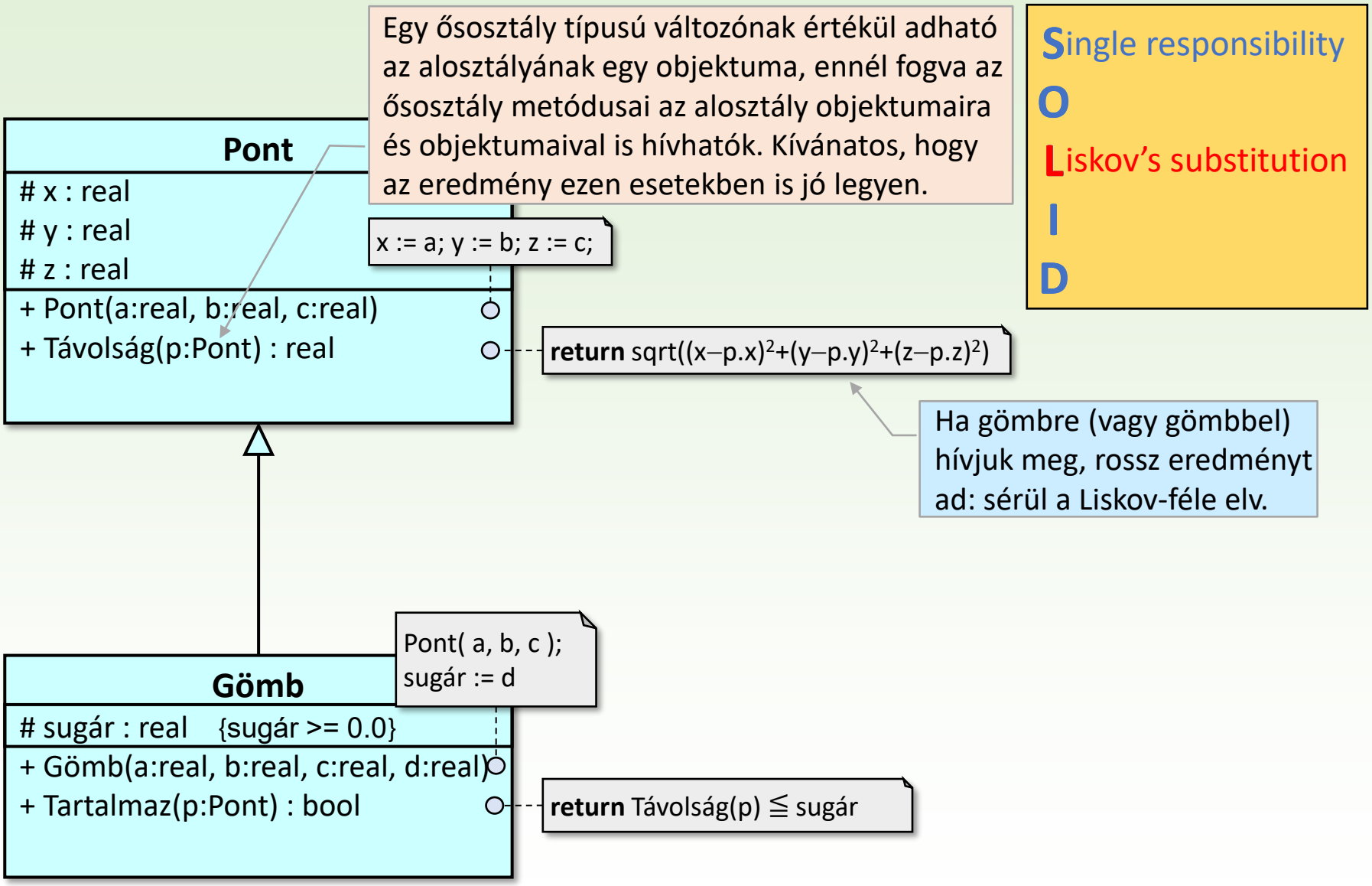
5

Az objektum-orientált nyelvek támogatják az öröklést: osztályok már meglévő osztályokból származtathatók.
Ősosztály változójának értékéül adható az alosztályának objektuma.

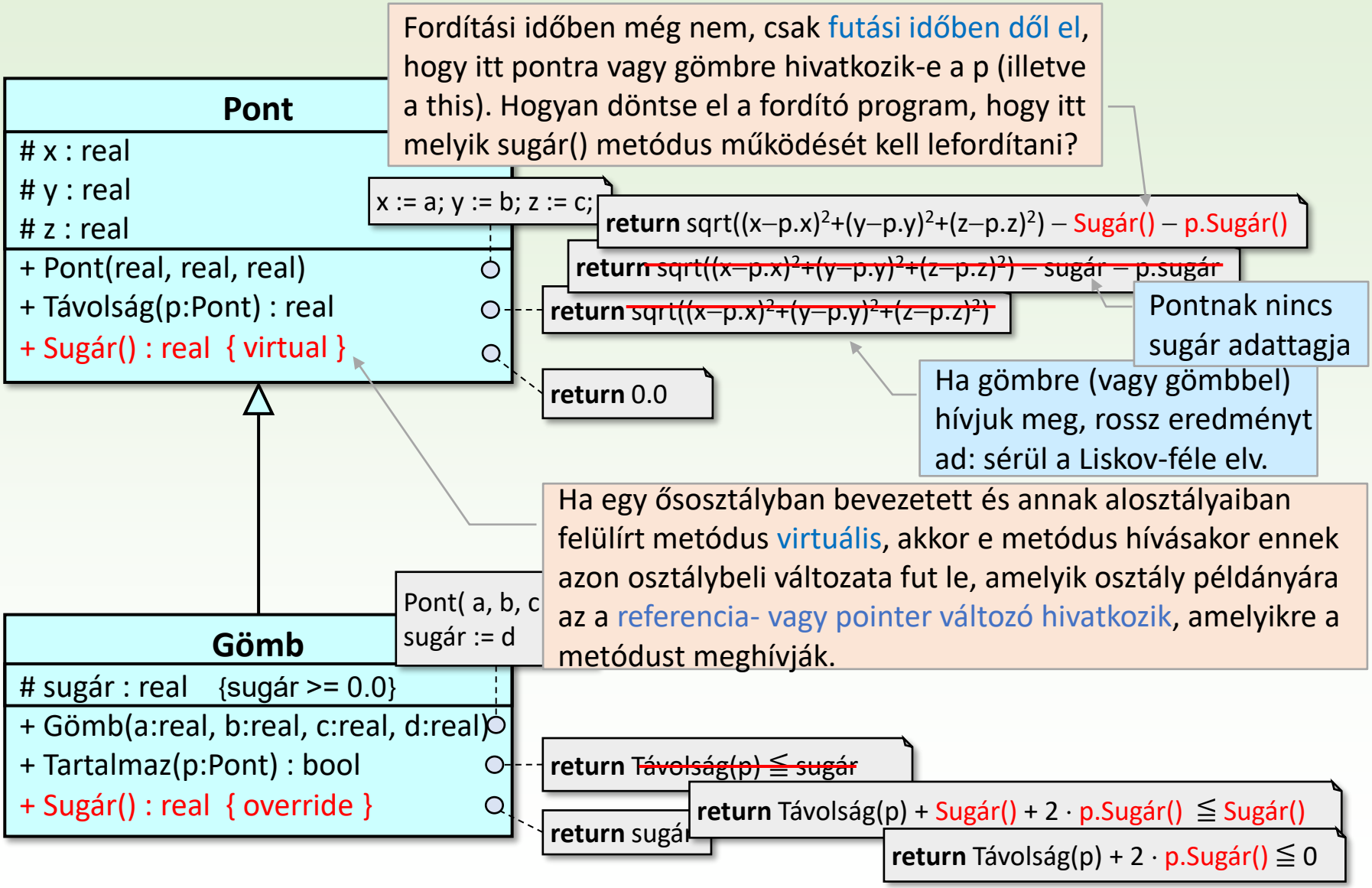
Származtatás és láthatóság

- Egy alosztály az ősosztály minden tagját örökli, de csak az ősosztály publikus és védett tagjaira hivatkozhat, a privát tagjaira nem, azokhoz csak indirekt módon, az ősosztálytól örökölt nem-privát metódusokkal férhet hozzá.
- A származtatás módja maga is lehet
  - publikus (*public*): ekkor az ősosztály publikus és védett tagjai az ősosztályban definiált láthatóságukkal együtt öröklődnek az alosztályra. (Az UML szerint ez a default, de a C++ nyelvben nem.)
  - védett (*protected*): ekkor az ősosztály publikus és védett tagjai mind védettek lesznek az alosztályban.
  - privát (*private*): ekkor az ősosztály publikus és védett tagjai mind privátok lesznek az alosztályban.

Példa: pontból gömb (2.)



Példa: pontból gömb (2.)



C++ : pontból gömb (2.)

```
Point  p(0,0,0);
Sphere g(1,1,1,1);

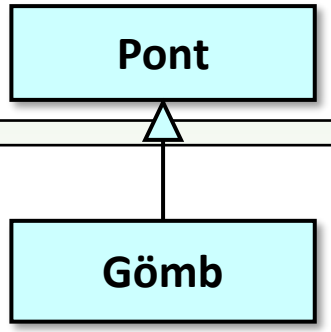
cout << p.distance(p) << endl;
cout << g.distance(p) << endl;
cout << p.distance(g) << endl;
cout << g.distance(g) << endl;
```

```
class Point {
protected:
        double _x, _y ,_z;
public:
        Point(double a, double b, double c) : _x(a), _y(b), _z(c) {}
        double distance(const Point &p) const
                { return sqrt(pow((_x-p._x),2)+pow((_y-p._y),2)+pow((_z-p._z),2))
                        - radius() - p.radius();}
        virtual double radius() const { return 0.0; }
};
```

referencia változó

this->radius()

virtuális metódus




```
class Sphere : public Point {
private:
        double _radius;
public:
        enum Errors{ILLEGAL_RADIUS};
        Sphere(double a, double b, double c, double d) : Point(a,b,c), _radius(d)
                { if (_radius<0.0) throw new IllegalRadiusException; }
        bool contains(const Point &p)const{return distance(p)+2*p.radius() <= 0;}
        double radius() const override { return _radius; }
};
```

felülírt metódus

Dinamikus altípusos polimorfizmus

- Ha egy ősosztály metódusát a leszármazott osztályban felülírjuk (override), akkor ez a metódus több alakkal is rendelkezik (polimorf).
- Mivel egy ősosztály típusú változónak mindig értékül adható az alosztályának egy példánya, ezért csak futási időben derülhet ki, hogy ez a változó az ősosztály egy példányára vagy alosztályának egy példányára hivatkozik-e. (késői vagy futási idejű vagy dinamikus kötés).
- Ha egy ősosztály típusú (referencia- vagy pointer) változóra egy polimorf virtuális metódust hívunk meg, akkor e metódusnak azon osztálybeli változata fut majd le, amilyen osztályú példányra hivatkozik a változó (dinamikus altípusos polimorfizmus).

6 Az objektum-orientált nyelvek ismérve a dinamikus altípusos polimorfizmus.